

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student Full Name	Nurailym Nurlankyzy
Student Identification Number	230103071
Date & Laboratory Session	18.09.2026
Workstation / Laptop Make & Model	Acer predator helios 16
Processor SKU / SoC Model	13th Gen Intel Core i7-13700HX
Core Topology (Physical P-cores / E-cores)	Physical Cores: 16 Logical SMT Threads: 24
Cache Hierarchy Capacity	L1: 1.4 KB L2: 14 MB L3: 30 MB
Host OS & Python Runtime	e.g., Linux Kernel 6.8 / Ubuntu 24.04 LTS / CPython 3.11.8

■ **MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE**

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ **EXECUTION PROTOCOL**

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline $T(1)$ and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

p = 1 (Baseline)	2.4963 s	1.00x	1.00x	100.0%
p = 2	1.6553 s	1.51×	2.00x	75.5%
p = 4	1.1715 s	2.13×	4.00x	53.3%
p = 8	0.9103 s	2.74×	8.00x	34.3%
p = 12	0.8902 s	2.80×	12.00x	23.3%
p = 16	1.0005	2.50×	16.00x	15.6%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

Student Response & Architectural Justification:

The observed speedup improves as the number of workers increases, but the improvement becomes very small after 8 workers. The best measured execution time was 0.8902 s with 12 workers. When the worker count increased to 16, the time increased to 1.0005 s and the speedup decreased from 2.80x to 2.50x.

Therefore, the empirical inflection point is approximately $p^* = 12$ workers. This indicates that the workload has reached physical CPU/resource saturation around this point. Adding more workers creates additional process scheduling and resource-contention overhead instead of providing useful speedup. My system reports 24 logical cores, so the 16-worker test is still below the total logical-thread count; therefore, the slowdown cannot be explained simply by exceeding 24 logical threads.

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

Student Response & Architectural Justification: Using my measured values:

$$T(1) = 2.4963 \text{ s}$$

$$T(4) = 1.1715 \text{ s}$$

First, calculate the measured speedup:

$$S(4) = T(1) / T(4)$$

$$S(4) = 2.4963 / 1.1715 = 2.1309$$

Using Amdahl's Law:

$$S(4) = 1 / [(1 - P) + P/4]$$

Therefore:

$$2.1309 = 1 / [(1 - P) + P/4]$$

$$1 / 2.1309 = 1 - P + P/4$$

$$0.4693 = 1 - 3P/4$$

$$3P/4 = 0.5307$$

$$P = (0.5307 \times 4) / 3$$

$$P \approx 0.7076$$

Therefore, the measured parallel fraction of the workload is approximately 70.76%.

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific

workload. (8 Points)

Student Response & Architectural Justification:

From Q1.2, the measured parallel fraction is:

$$P = 0.7076$$

Using the Amdahl asymptotic limit:

$$S_{\text{max}} = 1 / (1 - P)$$

$$S_{\text{max}} = 1 / (1 - 0.7076)$$

$$S_{\text{max}} \approx 3.42x$$

Therefore, the theoretical maximum speedup is approximately 3.42x. If a 128-core server were purchased for this specific workload, the workload would not achieve anything close to 128x speedup. According to the measured Amdahl fraction, the serial portion of the workload limits the maximum speedup to approximately 3.42x, regardless of how many additional cores are available.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements (16 * 8 bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.8175s	0.8925s	0.7649s	0.8175s	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.9158s	0.7957s	0.7461s	0.7957s	

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

Student Response & Architectural Justification:

In CPython, a list stores an array of `PyObject*` pointers. On a 64-bit build, `sizeof(void*) = 8` bytes.

A 64-byte cache line holds $64 / 8 = 8$ pointers.

If adjacent threads used unpadded indices 0, 1, 2, 3, all four pointers (0–3) sit within the same 8-pointer, 64-byte line — classic false-sharing setup.

With stride 16: indices used are 0, 16, 32, 48. Byte offsets: 0, 128, 256, 384. Since $128 > 64$, each accessed pointer lands in a different 64-byte line (in fact two lines apart), guaranteeing no two threads' target pointers ever share a line. General proof: for stride s pointers, the two indices are cache-line-safe whenever $s \times 8 \geq 64 \times 8$, i.e. $s \geq 8$. Stride 16 satisfies this with a full line of margin.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared → Modified → Invalid). (9 Points)

Student Response & Architectural Justification:

Initial state: the cache line holding indices 0–7 is loaded into Core 0's cache. State: **Exclusive (E)** — only Core 0 has it, unmodified.

Core 0 increments index 0: the line transitions **E → Modified (M)**. Core 0 now holds the only valid, dirty copy.

Core 1 wants to increment index 1 (same line): Core 1's cache controller issues a **Read-For-Ownership (RFO)** on the bus. Core 0 snoops this, must supply its dirty data (write-back or cache-to-cache transfer), and its own copy transitions **M → Invalid (I)**.

Core 1's copy becomes **Modified (M)** after it writes index 1.

Core 0 now wants to increment index 0 again: its line is Invalid, so it must issue its own RFO, forcing Core 1's line **M → I**, and Core 0 reloads and writes, becoming **M** again.

This **M→I→M→I** ping-pong repeats on every single increment across the four cores, each transition costing a full cache-line transfer over the interconnect (tens of cycles) instead of a cheap local write (~1 cycle).

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 `'alignas(64)'`, Java `'@Contended'`, or Go struct field padding). (8 Points)

Student Response & Architectural Justification:

C++11 `alignas(64)`: forces the compiler to align a struct or variable to a 64-byte boundary, guaranteeing each instance starts on its own cache line rather than relying on manual byte-counting/padding arrays.

Java `@Contended` (`sun.misc.Contended`, JEP 142): annotates a field so the JVM automatically inserts padding around it to isolate it on its own cache line, with the JVM managing the padding size rather than the programmer.

(Bonus, since you listed three options) **Go struct field padding via alignment:** Go doesn't have an automatic attribute like `@Contended`, but idiomatic Go uses explicit `_ [N]byte` padding fields or `//go:align` in newer proposals, and libraries like `golang.org/x/sys/cpu.CacheLinePad` provide a struct type specifically for this purpose so you don't hand-count bytes each time.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches

(futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time

TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```
class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")
```

Unsafe Accumulator (Race Condition)	2,000,000	2,000,000	0	0.0791	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	0.3821	4.83x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

Student Response & Architectural Justification:

My Unsafe Accumulator run produced a final count of 2,000,000/2,000,000 — no data corruption was observed on this execution. This does not mean the code is thread-safe; it means the race window was never hit on this particular run. `self.val += 1` still decomposes into three separate steps at the interpreter level — **LOAD** (read current `val`), **ADD** (compute `val + 1`), **STORE** (write back) — and a thread can still be preempted between **LOAD** and **STORE** by the GIL's periodic switch interval, causing another thread's concurrent **LOAD** to read a stale value and overwrite the first thread's update. The race condition exists structurally regardless of whether it manifested this run; it is probabilistic, governed by `sys.getswitchinterval()` and OS thread-scheduling timing, not deterministic. [If you reran and got a corrupted count on a later trial, report that exact number here as concrete evidence the race is real.]

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly **FASTER** than `LockedCounter` by at least 2.0x. Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over `LockedCounter`. (11 Points)

Student Response & Architectural Justification:

Lockless time: 0.0437s

Locked time: 0.3821s

Speedup = $0.3821 / 0.0437 = 8.74x$ — exceeds the required 2.0x threshold

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Student Response & Architectural Justification:

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```

import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()

```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	0.9480	1.00x	0.7586	1.00x
w = 2 Workers	1.0911	1.15x	0.9382	1.24x

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 4 Workers	1.5372	1.62x	2.8683	3.78x

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

Student Response & Architectural Justification:

1. Process-spawn overhead dominates at small worker counts. `mp.Pool(w)` forks/spawns `w` new OS processes each call, and on Windows this uses the `spawn` method (not `fork`), which re-imports the module and interpreter in each child — a fixed, non-trivial cost paid per worker regardless of workload. With only 4 workers and a modest 25M-iteration workload per worker, this fixed overhead is a larger fraction of total time than it would be with a heavier workload, which is why compute scaling isn't perfectly flat in your run.

2. Despite that shared overhead, the divergence between the two suites is still the real signal. Compute-bound work happens entirely in CPU registers/ALU — each worker process is self-contained and doesn't compete for a shared resource beyond spawn cost and CPU scheduling. Memory-bound work, by contrast, requires every worker to continuously stream data across the **same physical DRAM bus and memory controller**. As you add workers, they queue for that shared bus, and the marginal cost of each additional worker's array traffic compounds — hence memory time more than tripling at `w=4` while compute only grew by 62%. The compute suite is bounded by (imperfect, but comparatively cheap) per-process overhead; the memory suite is bounded by a genuinely shared, finite hardware resource that doesn't scale with core count.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

Student Response & Architectural Justification:

My machine has 2× 8GB SK Hynix DDR5-4800 modules (16GB total) installed in a dual-channel configuration (confirmed by separate memory controllers/channels in the CIM query output). Using the standard formula — $\text{Speed} \times \text{Bus Width} \times \text{Channels}$ — DDR5's 64-bit (8-byte) per-channel bus width gives: $4800 \text{ MT/s} \times 8 \text{ bytes} \times 2 \text{ channels} = \mathbf{76.8 \text{ GB/s}}$ theoretical peak bandwidth.

This finite, shared bandwidth directly explains my Task 4 memory-bound scaling results. All 4 worker processes must funnel their array reads/writes through this single 76.8 GB/s bus simultaneously — unlike compute-bound work, where each core's ALU/register operations don't compete for a shared resource. With `memory_heavy()` streaming a 50,000,000-element float64 array (~400MB) per worker, 4 concurrent workers attempt to push roughly 1.6GB total through the bus at once. Once aggregate demand approaches the 76.8 GB/s ceiling, workers begin queuing for bus access rather than executing in parallel, which is exactly why my `w=4` memory time (2.8683s) degraded 3.78x from baseline — far worse than the 1.62x degradation on the compute-bound suite, where no such shared bottleneck exists. Memory channel count and speed impose a hard ceiling on multi-core scalability that adding more cores cannot overcome once that ceiling is reached.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

Student Response & Architectural Justification:
OpenMP parallelizes across CPU cores that all still share one system memory bus — exactly the bottleneck your Task 4 data demonstrates. Adding OpenMP threads to a memory-bound embedding pipeline hits the same DRAM contention ceiling you just measured; more threads competing for the same bus doesn't help throughput.
GPUs pair compute cores with dramatically higher-bandwidth dedicated memory (GDDR6/HBM, often 300–2000+ GB/s vs. a typical laptop's ~40–90 GB/s system RAM) precisely because GPU workloads (including embedding/vector transforms) are usually memory-bandwidth-bound, not compute-bound. The hardware is architected for exactly the bottleneck your benchmark exposed. Since embedding transformations are largely streaming, low-arithmetic-intensity operations (read vector, do a small number of FLOPs, write vector — similar in kind to your `memory_heavy` function), they fall on the memory-bound side of the Roofline Model, where GPU bandwidth advantage translates directly into throughput gains that CPU core-count scaling cannot match.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
[] Task 3 Unsafe Corruption Verified [] Task 2 Cache Slowdown Verified [] Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

--	--	--

Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1d cachesize hw.l2 cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

count = 0

for i in range(n):

count += i * i

return count

Run both experiments with total_work = 50_000_000 split across T

workers # Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

- The Empirical Curve:** Plot $\$T\$$ (workers) on the X-axis vs. Execution Time on the Y axis for both threading.Thread and multiprocessing.Pool.
- The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
- The Architectural Autopsy:**
 - Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
 - Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD

Ryzen). How many *physical performance cores* do you have vs. *logical threads*? ○
Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.